Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings is an implemented structured peer-to-peer network specified by carriers, owner maps, protocol morphisms, and explicit obligations. Its motivating failure mode is authority collapse:

$$\mathsf{root}(id, data, route) \in \{\mathsf{server}, \mathsf{dns}, \mathsf{relay}, \mathsf{platform}\}.$$

Rings replaces that root with self-certifying identity and resource ownership over a CorrectChord overlay, while keeping cryptographic carriers disjoint from the routing carrier. The object studied here is

$$\mathsf{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X,H) &= (X, \mathrm{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\{D, V, M, H, Q, W\} \subset \mathrm{Obj}(\mathbf{Set}/R), \qquad \mathsf{CryptoCarrier} \cap R = \varnothing.$$

Claims are stratified as

$$\mathsf{Claim} = \mathsf{Spec} \cup \mathsf{Impl} \cup \mathsf{Evidence}_\Gamma \cup \mathsf{Obligation}.$$

I. Introduction

Centralized platforms are efficient deployment mechanisms but poor authority carriers: identity, naming, storage, and traffic control collapse to service operators [1]–[3]. Rings starts from the opposite requirement:

$$\begin{aligned} \mathsf{Own}(d) &= k_d, \\ \mathsf{Route}(x) &= \mathrm{owner}_N(H_\nu(x)), \\ \mathsf{Run}(\mathcal{P}_\nu) &= \tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \mathsf{RootOfAuthority} &\neq \mathsf{ApplicationServer}. \end{aligned}$$

The central modeling choice is carrier separation. The Chord identifier space is the additive cyclic group for placement and interval order; signatures, threshold schemes, encryption, and zero-knowledge protocols remain in their own fields or groups:

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathsf{CC} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}, \\ R \cap \mathsf{CC} &= \varnothing. \end{aligned}$$

This paper studies the algebra that makes the implementation coherent, then marks which claims are implemented, model checked in a finite scope, or still benchmark obligations.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; last updated July 4, 2026.

II. Problem Statement and Design Goals

Existing decentralized systems supply only parts of the required object:

$$\begin{aligned} \mathsf{Kademlia} &\Rightarrow \mathsf{lookup}_{XOR} \not\Rightarrow \mathsf{clockwiseOwner}, \\ \mathsf{RelayMixnet} &\Rightarrow \mathsf{pathPrivacy} \not\Rightarrow \mathsf{structuredOwner}, \\ \mathsf{Ledger} &\Rightarrow \mathsf{globalOrder} \not\Rightarrow \mathsf{localOverlay}. \end{aligned}$$

Obligation 1 (Design target).

$$\begin{aligned} G_{id} &= \mathsf{SelfCertID} \wedge \mathsf{DelegatedSession}, \\ G_{own} &= \mathsf{StructuredOwner}, \\ G_{io} &= \mathsf{BrowserNativeTransport}, \\ G_{eff} &= \mathsf{TypedProtocolEffect}, \\ G_{cr} &= \mathsf{CarrierCorrectCrypto}, \\ G_{rep} &= \mathsf{BoundedRepairEvidence}, \\ \mathsf{Rings} &\models G_{id} \wedge G_{own} \wedge G_{io} \wedge G_{eff} \wedge G_{cr} \wedge G_{rep}. \end{aligned}$$

Definition 1 (Methodology).

$$\begin{aligned} \mathsf{Method} &= \mathsf{Mot} \to \mathsf{Spec}_C \to \mathsf{Alg}_P \to \mathsf{ImplMap} \\ &\quad \to \mathsf{Evidence}_\Gamma \to \mathsf{Obligation}, \\ \Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}), \\ \mathsf{Admit}(c) &\Rightarrow c \in \mathsf{Spec} \cup \mathsf{Impl} \\ &\quad \cup \mathsf{Evidence}_\Gamma \cup \mathsf{Obligation}. \end{aligned}$$

The prose in this paper is used only where the construction needs motivation, scope, or comparison; the claims themselves are carried by definitions, constraints, invariants, algorithms, and obligations.

$$\mathsf{Contribution} = \{R, \mathbf{Set}/R, \Omega_N, \mathcal{R}_N, \mathsf{RepKey}^\rho, \mathsf{T}^{\mathcal{E}}_A, \mathsf{Explore}_\Gamma, \mathsf{Report}_{Rings}\}.$$

III. Algebraic System Model

Assumption 1 (System scope).

$$\begin{aligned} \mathsf{fault} &= \mathsf{fail\text{-}stop}, \\ \mathsf{auth}(m,\sigma,k) &= V(k,\sigma,m), \\ \mathsf{net} &= \mathsf{async}, \\ \mathsf{io} &= I_X : A_X^* \to \mathsf{IO}, \\ \mathsf{consensus} &\notin \mathsf{Overlay}, \\ \mathsf{order} &= \mathsf{SidecarWitness}, \\ \mathsf{crypto\_ops} &\cap \mathsf{route\_ops} = \varnothing. \end{aligned}$$

The fail-stop clause scopes overlay maintenance; identity, ranking, and application protocols still carry adversarial predicates at their own layer.

Definition 2 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x-a) \bmod M, \\ (a,b]_R &= \{x \in R : 0 < \delta_a(x) \le \delta_a(b)\}, \\ \mathsf{RouteOps}_R &= \{0, +, -, \delta_a, (\_,\_]_R\}, \\ \mathsf{Mult}_R &\notin \mathsf{RouteOps}_R. \end{aligned}$$

Definition 3 (Live topology).

$$\begin{aligned} N &\subset R, \quad N \neq \varnothing, \\ \mathrm{owner}_N(k) &= \arg\min_{u \in N} \delta_k(u), \\ \mathrm{pred}_N(n) &= \arg\min_{p \in N \setminus \{n\}} \delta_p(n), \\ \mathrm{Succ}^q_N(n) &= \mathrm{take}_q\big(\mathrm{sort}_{\delta_n}(N \setminus \{n\})\big), \\ \delta_a|_N &= \mathsf{injective}. \end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned} r &= |\mathrm{Succ}^r_N(n)|, \\ \mathsf{Steady}(N) &\Rightarrow |N| \ge r+1, \\ |N| < r+1 &\Rightarrow \mathsf{Init}(N), \\ \mathsf{Maintain} &= \{\mathsf{join}, \mathsf{rectify}\} \\ &\quad \cup \{\mathsf{stabilize}\}_{CorrectChord}. \end{aligned}$$

This is the CorrectChord stable base of Zave [4].

Definition 4 (Affine replica set).

$$\begin{aligned} \rho &\in \mathbb{N}_{>0}, \\ \mathrm{repkey}^\rho_i(k) &= k + \lfloor M \cdot i/\rho \rfloor, \\ \mathrm{RepKey}^\rho(k) &= \langle \mathrm{repkey}^\rho_i(k) : 0 \le i < \rho \rangle, \\ \mathrm{RepOwner}^\rho_N(k) &= \langle \mathrm{owner}_N(\mathrm{repkey}^\rho_i(k)) : 0 \le i < \rho \rangle. \end{aligned}$$

$$\begin{aligned} P_k &= \mathrm{set}(\mathrm{RepKey}^\rho(k)), \\ O_k &= \mathrm{set}(\mathrm{RepOwner}^\rho_N(k)), \\ |P_k| &= \rho, \\ |O_k| &\le \min(\rho, |N|), \\ \mathsf{collapse}_{N,\rho}(k) &\Leftrightarrow |O_k| < \rho. \end{aligned}$$

Thus replica keys specify redundancy intent; live owner images specify realized redundancy. CorrectChord successor lists remain the topology safety root.

Definition 5 (Rings overlay object).

$$\mathcal{R}_N = (R, N, \mathrm{owner}_N, \mathrm{pred}_N, \mathrm{Succ}^r_N, \mathrm{RepOwner}^\rho_N, F, E),$$

$$\begin{aligned} F &\subseteq N \times \mathbb{N} \times N, \\ E &= \textstyle\prod_{v \in VID} E_v, \\ \mathsf{SafetyRoot}(\mathcal{R}_N) &= (\mathrm{pred}_N, \mathrm{Succ}^r_N), \\ \mathsf{PerfWitness}(\mathcal{R}_N) &= F. \end{aligned}$$

Proposition 1 (Rotation equivariance).

$$t \in R, \quad N + t = \{n + t : n \in N\} \Rightarrow \mathrm{owner}_{N+t}(k+t) = \mathrm{owner}_N(k) + t.$$

Proof.

$$\begin{aligned} \forall n \in N. \quad \delta_{k+t}(n+t) &= ((n+t) - (k+t)) \bmod M \\ &= \delta_k(n). \end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned} \mathsf{PlacementOps} &= \{0, +, -, \delta_a, (a,b]_R\} \\ &\quad \cup \{\mathrm{owner}_N\} \\ &\quad \cup \{\mathrm{RepKey}^\rho, \mathrm{RepOwner}^\rho_N\}, \\ \mathsf{CryptoOps} &= \{\times, ^{-1}, \cdot_{scalar}, \mathsf{interp}\} \\ &\quad \cup \{\mathsf{pair}, \mathsf{subgroup}\}, \\ \mathsf{PlacementOps} &\cap \mathsf{CryptoOps} = \varnothing, \\ \mathsf{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}. \end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \to R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	group operation plus byte-to-point adapter
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table I
Carrier boundaries in the Rings model

IV. Architecture As Interfaces

Definition 6 (Layer tuple).

$$\mathcal{A} = (I, T, O, X, P, L)$$

I	$(DID, \Pi, \mathrm{Session})$
T	$(C, A_T, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$
O	$\mathcal{R}_N$
X	$(S_X, A_X, \mathsf{cap}, I_X)$
P	$\{\mathcal{P}_\nu\}_{\nu \in Namespace}$
L	$\mathsf{App} \circ P$

Definition 7 (Identity proof).

$$\Pi = (K, \Sigma, V, D, \mathsf{enc}, \mathsf{ttl})$$

$$\begin{aligned} V &: K \times \Sigma \times B^* \to \{\top, \bot\}, \\ D &: K \to R, \\ \mathsf{enc} &: \Sigma \to B^*, \\ \mathsf{ttl} &: \Sigma \to Time. \end{aligned}$$

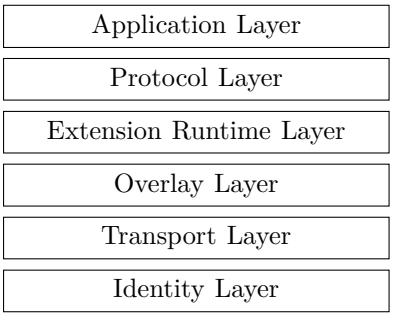

Figure 1. Layers of Rings Network

Invariant 1 (Session authority).

$$d \in X_D, \qquad s \in K.$$

$$\text{Session}(d, s, e) \Rightarrow V(k_d, \sigma_{d \to s}, d \| s \| e) = \top \wedge now < e.$$

$$\mathsf{HopCheck}(d, s, e) \equiv \text{Session}(d, s, e).$$

Definition 8 (Transport interface).

$$T = (C, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$$

$$A_T = \{\mathsf{Offer}, \mathsf{Answer}, \mathsf{Ice}, \mathsf{Open}, \mathsf{Send}, \mathsf{Recv}, \mathsf{Close}\}.$$

$$I_T : A_T^* \to \mathsf{IO}_{WebRTC} \cup \mathsf{IO}_{Native}.$$

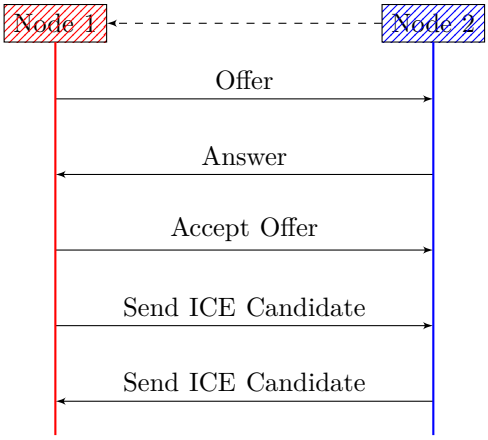

Figure 2. Standard SDP/ICE exchange

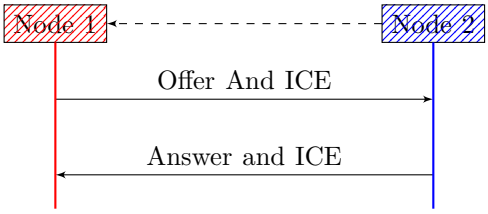

Figure 3. Rings single round-trip SDP/ICE exchange

Definition 9 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \mathsf{cap}, I_X)$$

$$\begin{aligned} \mathsf{cap} &\subseteq A_X, \\ I_X &: A_X^* \to \mathsf{IO}, \\ I_X(\epsilon) &= \mathsf{Noop}, \\ I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta). \end{aligned}$$

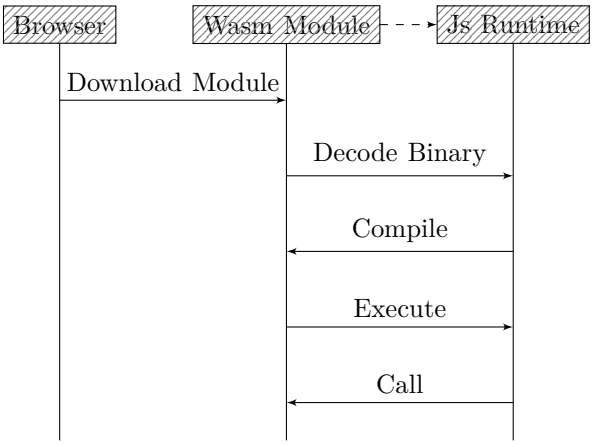

Figure 4. Wasm module execution boundary

Obligation 2 (Runtime confinement).

$$\begin{aligned} \tau &: S \times I \to \mathcal{E} + (S \times O \times A_X^*), \\ \mathsf{emit}(\tau(s, i)) &\subseteq \mathsf{cap}, \\ a \notin \mathsf{cap} &\Rightarrow I_X(a) = \mathsf{Reject}(a). \end{aligned}$$

V. Category and Effects

Definition 10 (Namespace object).

$$\begin{aligned} \nu &\in \{D, V, M, H, U, Q, W\}, \\ H_\nu &: X_\nu \to R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 11 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \to \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \text{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \to N.$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \to X_\mu, \qquad H_\mu \circ f = H_\nu.$$

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\text{owner}_N \circ H_\mu \circ f = \text{owner}_N \circ H_\nu.$$

□

Definition 12 (Writer effect monad).

$$\begin{aligned} A^* &= \mathsf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg\!= f &= \mathsf{let}\ f(x) = (y, \beta)\ \mathsf{in}\ (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^\star \circ g^\star) \circ f^\star = h^\star \circ (g^\star \circ f^\star).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

Definition 13 (Typed failure stack).

$$\mathsf{T}_A^{\mathcal{E}}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \mathsf{RecoverableReport} &\in A. \end{aligned}$$

Definition 14 (Protocol object).

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu)$$

$$\tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Definition 15 (Protocol morphism).

$$\Sigma : \mathcal{P}_\nu \to \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$$

$$\begin{aligned} \sigma_A^* &: A_\nu^* \to A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

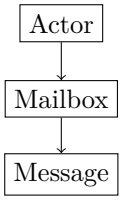

Figure 5. Actor Model

VI. Chord Overlay

$$\begin{aligned} \mathsf{Base} &= \mathsf{Chord}, \\ \mathsf{Maintenance} &= \mathsf{CorrectChord}, \\ \mathsf{Object} &= \mathcal{R}_N, \\ \mathsf{Witnesses} &= \{\text{Algorithms } 1, 2, 3\}. \end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 16 (Local node state).

$$S_n = (p_n, L_n, F_n, E_n)$$

$$\begin{aligned} p_n &\in N \cup \{\bot\}, \\ L_n &\in N^{\leq r}, \\ F_n &: \mathbb{N} \rightharpoonup N, \\ E_n &\subseteq E, \\ p_n = \mathrm{pred}_N(n), &\qquad L_n = \mathrm{Succ}_N^r(n). \end{aligned}$$

Definition 17 (Finger target).

$$\begin{aligned} \mathrm{target}_i(n) &= n + 2^i, \\ \mathrm{finger}_i(n) &= \mathrm{owner}_N(\mathrm{target}_i(n)), \\ F_n(i) &\in \{\bot, \mathrm{finger}_i(n)\}. \end{aligned}$$

Invariant 2 (Ring fixpoint).

$$\mathrm{Inv}_{ring}(N, S) \equiv \forall n \in N.\ p_n = \mathrm{pred}_N(n) \wedge L_n = \mathrm{Succ}_N^r(n).$$

□

Algorithm 1 DHT Join Transitions

```
1: function BeginJoin(n, known)
2:     pred[n] ← none
3:     a ← FindSuccessor(n.did)
4:     return Remote(known, a)
5: function InstallSuccessor(n, s)
6:     succ[n] ← take(r, [s])
7:     return Remote(s, QuerySuccList(s))
8: function InstallSuccList(n, s, list)
9:     succ[n] ← take(r, s :: list)
10:    if head(succ[n]) ≠ none then
11:        h ← head(succ[n])
12:        notify ← Notify(h, n.did)
13:        sync ← SyncStorage(h)
14:        return Multi(notify, sync)
15:    else
16:        return Noop
```

Algorithm 2 DHT Lookup Algorithm

```
1: function Lookup(key, node)
2:     s ← head(succ[node])
3:     if s = none then
4:         return Repair(QuerySuccList(node))
5:     if key ∈ (node.did, s] then
6:         return Local(s)
7:     next ← closest_preceding_node(node, key)
8:     if next = none then
9:         return Repair(QuerySuccList(s))
10:    else
11:        a ← FindSuccessor(key)
12:        return Remote(next, a)
```

Invariant 3 (Lookup owner preservation).

$$\mathsf{Steady}(N) \wedge \mathrm{Inv}_{ring}(N, S) \Rightarrow \mathrm{Lookup}(S, k) \Downarrow \mathrm{owner}_N(k).$$

$$\Downarrow = \mathsf{Reach}_{A_D}(2, 3).$$

VII. DID and Resource Algebra

Definition 18 (DID namespace).

$$\mathcal{D} = (X_D, H_D, \Pi, \mathrm{Session}), \qquad H_D : X_D \to R.$$

$$\begin{aligned} D(k) &= H_D(k), \\ \mathrm{owner}_N(D(k)) &\in N, \\ \mathrm{Session} &\subseteq X_D \times K \times Time. \end{aligned}$$

Definition 19 (VID namespace).

$$\begin{aligned} \mathcal{V} &= (X_V, H_V, \mathrm{Auth}_V), \\ H_V &: X_V \to R, \\ \mathrm{PrivateKey}(H_V(x)) &= \bot, \\ \mathrm{Auth}_V(op, x, \sigma, caps) &\in \{\top, \bot\}. \end{aligned}$$

Definition 20 (Biased order).

$$A \leq_C B \iff \delta_C(A) \leq \delta_C(B).$$

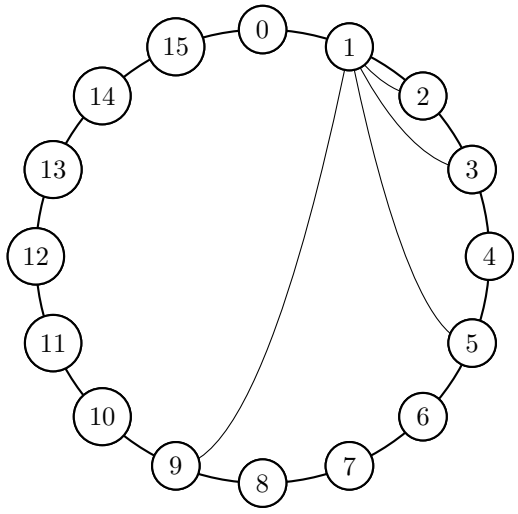

Figure 6. Chord finger targets from node 1 in a 16-node ring

Algorithm 3 DHT Stabilization

```
procedure Stabilize(n, topo)
    actions ← []
    c ← topo.pred
    s ← head(succ[n])
    if c ≠ none and c ∈ (n.did, s) then
        succ[n] ← insert(succ[n], c)
        actions ← actions :: FindSuccessor(c)
    succ[n] ← take(r, merge(succ[n], topo.succ))
    if head(succ[n]) ≠ none then
        h ← head(succ[n])
        actions ← actions :: Notify(h, n.did)
    return Multi(actions)
```

Definition 21 (Replicated entry algebra).

$$\begin{aligned} \mathcal{E}_v^{join} &= (E_v, \sqsubseteq, \sqcup, \bot), \\ \mathcal{E}_v^{final} &= (E_v, \mathsf{conflict}_v, \mathsf{finalize}_v). \end{aligned}$$

$$\begin{aligned} x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \bot &= x, \\ \mathsf{finalize}_v &: \mathsf{conflict}_v(E_v) \to E_v. \end{aligned}$$

$$\mathsf{case}_1 = \mathsf{CRDT}.$$

Algorithm 4 Biased Circular Comparison

```
function BiasedCompare(A, B, C)
    a ← δ_C(A)
    b ← δ_C(B)
    if a < b then
        return A <_C B
    else if a > b then
        return B <_C A
    else
        return A = B
```

This is the CRDT join-semilattice case [6].

Invariant 4 (Placement).

$$\begin{aligned} K_{\nu,x} &= \text{replicaKeys}(H_\nu(x)), \\ P_{\nu,x} &= \text{set}(\text{RepKey}^\rho(H_\nu(x))), \\ \text{Inv}_{place} &\equiv \forall \nu, x.\ K_{\nu,x} \subseteq P_{\nu,x}. \end{aligned}$$

Obligation 3 (Replica realization).

$$\begin{aligned} O_{\nu,x} &= \text{set}(\text{RepOwner}_N^\rho(H_\nu(x))), \\ L_{\nu,x} &= |O_{\nu,x}|, \\ \mathsf{FullRed}_{\nu,x} &\Leftrightarrow L_{\nu,x} = \min(\rho, |N|), \\ \mathsf{Collapse}_{\nu,x} &\Leftrightarrow L_{\nu,x} < \min(\rho, |N|), \\ \mathsf{LiveClaim}_{\nu,x}(\rho) &\Rightarrow \mathsf{FullRed}_{\nu,x}, \\ \mathsf{Report}_{Rings} &\ni |\{(\nu, x) : \mathsf{Collapse}_{\nu,x}\}|. \end{aligned}$$

Definition 22 (Mailbox).

$$\begin{aligned} mbox(d, e) &= H_M(\texttt{rings.mailbox} \| d \| e), \\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 23 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay, \\ &\quad Cap, K, exp, rank), \\ VID(h) &= H_H(h), \\ \mathsf{Lookup}(h) &= P_H^N(h), \\ \mathsf{Valid}(h, \sigma) &= \text{Auth}_H(h, \sigma). \end{aligned}$$

Definition 24 (Subring).

$$\mathcal{S}_u = (u, N_u, \text{owner}_{N_u}, \text{Succ}_{N_u}^{r_u}, E_u)$$

$$\begin{aligned} u &= H_U(policy), \\ memberlog_u &\in \mathsf{MergeLog} \cup \mathsf{FinalState}. \end{aligned}$$

VIII. Cryptographic Carriers and Traffic

Definition 25 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q, \\ \mathsf{Scalar}(\mathbb{G}) &= \mathbb{Z}_q, \\ x &\in \mathbb{Z}_q, \\ h &= g^x, \\ \mathbb{G} &\neq R. \end{aligned}$$

Algorithm 5 ElGamal Encryption and Decryption

```
Input: Encoded group element m_G ∈ 𝔾, private key
  x ∈ ℤ_q, public key h = g^x, generator g
Output: Group ciphertext (c_1, c_2)
procedure Encryption
    Choose a random integer k ∈ ℤ_q
    Compute c_1 = g^k
    Compute c_2 = m_G · h^k
    return (c_1, c_2)
procedure Decryption
    Compute m_G = c_2 · (c_1)^{-x}
    return m_G
```

Algorithm 5 is the group-level carrier operation. Byte payloads must first be encoded into curve points and split into bounded frames. The direct ElGamal body is malleable when detached from its signed transport

envelope; Rings therefore treats message signatures, DID ownership of the public key, stream identifiers, sequence numbers, and final-frame checks as part of the security boundary rather than as optional metadata.

Definition 26 (Signed encrypted frame).

$$f = (sid, seq, last, cipher, pk_s, \sigma_s).$$

$$\begin{aligned} m_f &= sid \| seq \| last \| cipher, \\ \text{FrameValid}_\Delta(f) &\Leftrightarrow V(pk_s, \sigma_s, m_f) = \top \\ &\quad \wedge\ \text{Session}(D(pk_s), sid, e) \\ &\quad \wedge\ \text{SeqOK}_\Delta(seq, last). \end{aligned}$$

Definition 27 (Receiver sequence policy).

$$\Delta = (next, final, closed, pending, seen, w).$$

$$\begin{aligned} \mathsf{NoFuture}_\Delta(seq) &\Leftrightarrow \neg\exists p \in pending.\ p > seq, \\ \mathsf{Dup}_\Delta(seq) &\Leftrightarrow seq \in seen \cup pending, \\ \mathsf{New}_\Delta(seq, last) &\Leftrightarrow seq \notin seen \cup pending \\ &\quad \wedge \neg closed \\ &\quad \wedge seq - next \le w \\ &\quad \wedge (final = \bot \vee seq \le final) \\ &\quad \wedge (last \Rightarrow \mathsf{NoFuture}_\Delta(seq)), \\ \text{SeqOK}_\Delta(seq, last) &\Leftrightarrow \mathsf{Dup}_\Delta(seq) \\ &\quad \vee \mathsf{New}_\Delta(seq, last), \\ \mathsf{Dup}_\Delta(seq) &\Rightarrow \mathsf{emit} = \epsilon. \end{aligned}$$

Here w is receiver policy, not a signed wire field.

Definition 28 (Secret sharing placement).

$$\begin{aligned} P &\in \mathbb{F}_q[X], \\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2, \\ H_S(sh_i) &\in R, \\ \text{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 29 (Relay frame).

$$r = (path, dst, \nu, session, payload, report).$$

$$\begin{aligned} \text{RelayValid}(r) &= \text{SessionValid}(session) \\ &\quad \wedge\ dst \in R, \\ \text{SplitVocabulary} &= \mathsf{MSRP}. \end{aligned}$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```
procedure E2E Chunking
    Input: data, chunk size
    Output: chunks
    chunks ← []
    total-chunks ← ceil(len(data)/chunk-size)
    for i in range(total-chunks) do
        chunk ← data[i*chunk-size : (i+1)*chunk-size]
        append chunk to chunks
    return chunks
```

Invariant 5 (Chunk reassembly).

$$\begin{aligned} \text{complete}(m) \Leftrightarrow \forall i < total(m).\ \exists! c_i \\ \wedge \sum_i |c_i| \le B_m \\ \wedge\ now < ttl(m). \end{aligned}$$

IX. Ordering, Ranking, and Security

Definition 30 (Sidecar witness).

$$w = (domain, root, height, time, finality).$$

$$\begin{aligned} c &= H(msg), \\ \text{Witness}(msg, w) &\Leftrightarrow \text{Include}(c, w.root) = \top \\ &\quad \wedge \text{Final}(w) = \top. \end{aligned}$$

Definition 31 (Witness order).

$$\begin{aligned} a \prec b \Leftrightarrow domain(w_a) = domain(w_b) \\ \wedge\ time(w_a) < time(w_b). \end{aligned}$$

$$\chi : Domain_a \times Domain_b \to Order.$$

Definition 32 (Ranking event).

$$\begin{aligned} q = (subject, rater, behavior, \\ evidence, epoch, weight, \sigma). \end{aligned}$$

$$\begin{aligned} m_q &= subject \| behavior \| evidence \| epoch, \\ \text{RankValid}(q) &\Leftrightarrow V(k_{rater}, \sigma, m_q) = \top. \end{aligned}$$

Definition 33 (Score aggregation).

$$\begin{aligned} score_e(s) = \text{robust}\{weight(q) \cdot value(q) : \\ q.subject = s\}. \end{aligned}$$

$$\text{robust} \in \{\mathsf{median}, \mathsf{trimmedMean}, \mathsf{declared}\}.$$

Attack	Predicate	Required boundary
Sybil	many DIDs, one actor	admission and ranking policy
Replay	stale valid σ	TTL, nonce cache, stream/final
MITM	wrong key for DID	DID proof and signed envelope
DoS	unbounded resource use	quotas and chunk bounds

Table II
Security predicates and required controls

$$\begin{aligned} \mathsf{Base} &\not\vdash \mathsf{SybilResist} \wedge \mathsf{Anon} \wedge \mathsf{DoSResist}, \\ \mathsf{SecClaim} &\Rightarrow \mathsf{Adm} \wedge \mathsf{Rank} \wedge \mathsf{Relay} \wedge \mathsf{Quota}. \end{aligned}$$

Obligation 4 (Replay rejection).

$$\begin{aligned} \text{accept}(s, \eta, t) &\Rightarrow \eta \notin Seen_s \wedge t < exp(s), \\ Seen'_s &= Seen_s \cup \{\eta\}, \\ \mathsf{emitDelivery} &< Seen'_s. \end{aligned}$$

X. Verification Evidence and Evaluation Scope

$$\mathsf{Spec} \neq \mathsf{Evidence}_\Gamma \neq \mathsf{Evaluation}.$$

The repository contains executable evidence for finite overlay, storage, and message-flow obligations; it does not yet contain a wide-area Rings benchmark.

Definition 34 (State relation).

$$C = (S, Q)$$

$$\frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S, i) = (S', O, \beta)}{(S, Q) \to (S', Q_0 \cdot \beta)}$$

$$\mathsf{Explore}_\Gamma = \mathsf{ModelCheck}(\to, \Gamma)$$

$$\begin{aligned} \Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}), \\ \mathsf{ModelClaim}(\Gamma) &\Rightarrow \mathsf{finite}(\Gamma) \wedge \mathsf{declared}(\Gamma). \end{aligned}$$

The relation follows TLA+ style and Stateright execution in the declared finite scope [9], [10].

Invariant 6 (Effect refinement).

$$\mathrm{Impl}_A(s, i) \preceq I_A^\sharp(\tau(s, i)).$$

$$\pi_{state}(\mathrm{Impl}_A) = \pi_{state}(I_A^\sharp \circ \tau).$$

Obligation 5 (Topology coverage).

$$\mathcal{L} = \{line, wrap, gap, partition, merge, churn\}$$

$$\mathcal{S} = \{loss, dup, reorder, timeout, restart\}.$$

$$\mathsf{CheckCase} = (L, S, P), \qquad L \in \mathcal{L},\ S \in \mathcal{S}.$$

Artifact	Checked scope	Limitation
Topology tests	join, rectify, stabilize fixpoints on finite DID layouts	not a parametric TLAPS proof
Stateright	predecessor update, small discovery abstraction, storage CRDT, handoff	scoped models, not a full live network
Trace replay	deterministic delivery schedules over real routing code	not an Internet latency benchmark
E2E tests	signed direct-ElGamal frame order, final marker, and replay behavior	not a symbolic cryptographic proof

Table III
Verification evidence and explicit limits

Definition 35 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness}.$$

$$h = O(\log |N|) \quad (F_n \text{ populated}).$$

Definition 36 (Reliability baseline).

$$\begin{aligned} failrate &= \frac{|F|}{|Q|}, \\ timeoutMean &= \frac{1}{|Q|} \sum_{q \in Q} to(q). \end{aligned}$$

Obligation 6 (Benchmark report).

$$\begin{aligned} \mathsf{Report}_{Rings} = (&N, \rho, r, \mathsf{transport}, \mathsf{runtime}, \\ &\mathsf{scheduler}, T_{total}, timeoutMean, failrate). \end{aligned}$$

$$\mathsf{Baseline}_{Chord} \not\equiv \mathsf{Measurement}_{Rings}.$$

$$\begin{aligned} \vec{T} &= (T_{conn}, T_{hop}, T_{crypto}, T_{app}, T_{witness}), \\ \mathsf{PerfClaim}_R &\Rightarrow \mathsf{Measure}_{build}(\vec{T}), \\ \mathsf{Baseline}_C &= \mathsf{hop/churn}. \end{aligned}$$

Chord remains the overlay baseline [5]; Rings performance requires Rings-specific measurements.

XI. Related Work

Use the comparison coordinates

$$\mathsf{Axis}(S) = (\mu, \omega, \chi, \lambda).$$

$$\begin{aligned} \mu &= \mathsf{metric}, \\ \omega &= \mathsf{owner}, \\ \chi &= \mathsf{order}, \\ \lambda &= \mathsf{runtime}. \end{aligned}$$

S	$\mathsf{Axis}(S)$
Kad	$(XOR, \mathsf{bucket}, \bot, \mathsf{native})$
Chord	$(\mathsf{circ}, \mathsf{succ}, \bot, \mathsf{native})$
Tor/Nym	$(\mathsf{path}, \bot, \bot, \mathsf{relay})$
Ledger	$(\bot, \mathsf{acct}, \mathsf{global}, \mathsf{repl})$
Rings	$(\mathsf{circ}, \mathrm{owner}_N, \mathsf{sidecar}, \mathsf{web} \cup \mathsf{native})$

Kademlia/IPFS/libp2p supply scalable lookup but not the clockwise owner law needed here [11]–[13]. Tor and Nym supply relay-path privacy, not structured ownership [14], [15]. Ledgers supply global order; Rings keeps ordering outside the base overlay. WebRTC and WebAssembly are runtime adapters, so the same protocol object is interpreted in browser and native environments [16], [17].

System	Metric	Ownership law
Kademlia	XOR	bucket routing
Relay mixnets	path	no structured owner
Rings	circular	clockwise owner, successor list, affine placement keys

Table IV
Routing and ownership comparison [11]–[15]

XII. Conclusion

The paper's claim is the following structured object, not an informal bundle of network mechanisms:

$$\mathsf{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned} \mathsf{Overlay} &= \mathsf{CorrectChord}(R, N, r), \\ \mathsf{Resource} &= (X_\nu, H_\nu) \in \mathbf{Set}/R, \\ \mathsf{Placement} &= \Omega_N(X_\nu, H_\nu), \\ \mathsf{Transition} &= \tau_\nu : S_\nu \times I_\nu \to Y_\nu, \\ Y_\nu &= \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \mathsf{Crypto} &= \mathbb{G} \vee \mathbb{F}_q, \quad \mathsf{Crypto} \cap R = \varnothing, \\ \mathsf{Evidence} &= \{\mathsf{scopedModels}, \mathsf{traceReplay}\} \\ &\quad \cup \{\mathsf{invariants}, \mathsf{benchmarkObligations}\}. \end{aligned}$$

References

[1] N. Kshetri, "Privacy and security issues in cloud computing: The role of institutions and institutional evolution," Telecommunications Policy, vol. 37, no. 4–5, pp. 372–386, 2013.

[2] D. Zissis and D. Lekkas, "Addressing cloud computing security issues," Future Generation Computer Systems, vol. 28, no. 3, pp. 583–592, 2012.

[3] N. Kshetri and S. Murugesan, "Cloud computing and EU data privacy regulations," Computer, vol. 46, no. 3, pp. 86–89, 2013.

[4] P. Zave, "Reasoning about identifier spaces: How to make Chord correct," IEEE Transactions on Software Engineering, vol. 43, no. 12, pp. 1144–1156, 2017, https://arxiv.org/abs/1610.01140.

[5] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications. ACM, 2001, pp. 149–160.

[6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[7] A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[8] B. Campbell, R. Mahy, and C. Jennings, "The message session relay protocol (MSRP)," RFC 4975, 2007, https://www.rfc-editor.org/info/rfc4975.

[9] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[10] J. Nadal, "Stateright: Model checking for implementing distributed systems," https://www.stateright.rs/, accessed: July 4, 2026.

[11] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.

[12] J. Benet, "IPFS - content addressed, versioned, P2P file system," arXiv:1407.3561 [cs.NI], 2014, https://arxiv.org/abs/1407.3561.

[13] Protocol Labs, "libp2p: A modular network stack," 2026, https://libp2p.io/.

[14] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The Second-Generation onion router," in 13th USENIX Security Symposium (USENIX Security 04). San Diego, CA: USENIX Association, Aug. 2004, https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router.

[15] C. Diaz, H. Halpin, and A. Kiayias, "The Nym network: The next generation of privacy infrastructure," Whitepaper, 2021, https://nym.com/nym-whitepaper.pdf.

[16] H. Alvestrand, "Overview: Real-time protocols for browser-based applications," RFC 8825, 2021, https://www.rfc-editor.org/rfc/rfc8825.

[17] WebAssembly Working Group, "WebAssembly core specification," W3C Recommendation, 2019, https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/.